

Introducción

Un patrón de diseño describe una situación o problema habitual que nos encontramos durante el desarrollo de una aplicación, así como la forma en la que este debe ser resuelto.

El grupo de cuatro desarrolladores GoF, elaboraron un catálogo que recoge 23 patrones de diseño. Este conjunto de patrones están clasificados en tres grandes grupos:

Creacionales.

Tratan sobre la forma de crear objetos, abstrayendo de la manera en la que estos son creados e inicializados. Entre los principales patrones creacionales tenemos:

- **Abstract Factory:** Nos provee una interfaz que delega la creación de un conjunto de objetos relacionados sin necesidad de especificar en ningún momento cuáles son las implementaciones concretas.
- **Factory Method:** Expone un método de creación, delegando en las subclases la implementación de este método.
- **Builder:** Separa la creación de un objeto complejo de su estructura, de tal forma que el mismo proceso de construcción nos puede servir para crear representaciones diferentes.
- **Singleton:** limita a uno el número de instancias posibles de una clase en nuestro programa, y proporciona un acceso global al mismo.
- **Prototype:** Permite la creación de objetos basados en “plantillas”. Un nuevo objeto se crea a partir de la clonación de otro objeto.

Estructurales.

Establecen cómo como combinar clases y objetos para formar nuevas estructuras y proporcionar nuevas funciones. Entre los más importantes están:

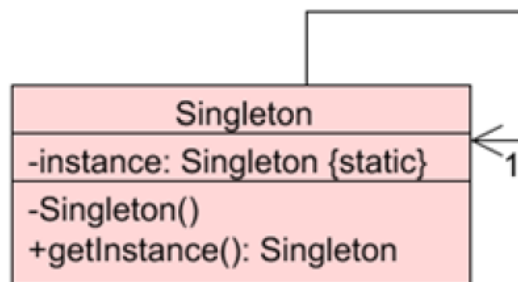
- **Adapter:** Permite a dos clases con diferentes interfaces trabajar entre ellas, a través de un objeto intermedio con el que se comunican e interactúan.
- **Bridge:** Desacopla una abstracción de su implementación, para que las dos puedan evolucionar de forma independiente.
- **Composite:** Facilita la creación de estructuras de objetos en árbol, donde todos los elementos emplean una misma interfaz. Cada uno de ellos puede a su vez contener un listado de esos objetos, o ser el último de esa rama.
- **Decorator:** Permite añadir funcionalidad extra a un objeto (de forma dinámica o estática) sin modificar el comportamiento del resto de objetos del mismo tipo.
- **Facade:** Una facade (o fachada) es un objeto que crea una interfaz simplificada para tratar con otra parte del código más compleja, de tal forma que simplifica y aísla su uso. Un ejemplo podría ser crear una fachada para tratar con una clase de una librería externa.
- **Flyweight:** Una gran cantidad de objetos comparte un mismo objeto con propiedades comunes con el fin de ahorrar memoria.
- **Proxy:** Es una clase que funciona como interfaz hacia cualquier otra cosa: una conexión a Internet, un archivo en disco o cualquier otro recurso que sea costoso o imposible de duplicar.

De comportamiento

Centrados en los algoritmos y en el reparto de responsabilidades entre las clases para, entre otras cosas, evitar el acoplamiento entre los objetos. La mayor parte de patrones pertenece a este grupo:

- **Command:** Son objetos que encapsulan una acción y los parámetros que necesitan para ejecutarse.
- **Chain of responsibility:** se evita acoplar al emisor y receptor de una petición dando la posibilidad a varios receptores de consumirlo. Cada receptor tiene la opción de consumir esa petición o pasárselo al siguiente dentro de la cadena.
- **Interpreter:** Define una representación para una gramática así como el mecanismo para evaluarla. El árbol de sintaxis del lenguaje se suele modelar mediante el patrón Composite.
- **Iterator:** Se utiliza para poder movernos por los elementos de un conjunto de forma secuencial sin necesidad de exponer su implementación específica.
- **Mediator:** Objeto que encapsula cómo otro conjunto de objetos interactúan y se comunican entre sí.
- **Memento:** Este patrón otorga la capacidad de restaurar un objeto a un estado anterior.
- **Observer:** Los objetos son capaces de suscribirse a una serie de eventos que otro objetivo va a emitir, y serán avisados cuando esto ocurra.
- **State:** Permite modificar la forma en que un objeto se comporta en tiempo de ejecución, basándose en su estado interno.
- **Strategy:** Permite la selección del algoritmo que ejecuta cierta acción en tiempo de ejecución.
- **Template Method:** Especifica el esqueleto de un algoritmo, permitiendo a las subclasses definir cómo implementan el comportamiento real.
- **Visitor:** Permite separar el algoritmo de la estructura de datos que se utilizará para ejecutarlo. De esta forma se pueden añadir nuevas operaciones a estas estructuras sin necesidad de modificarlas.

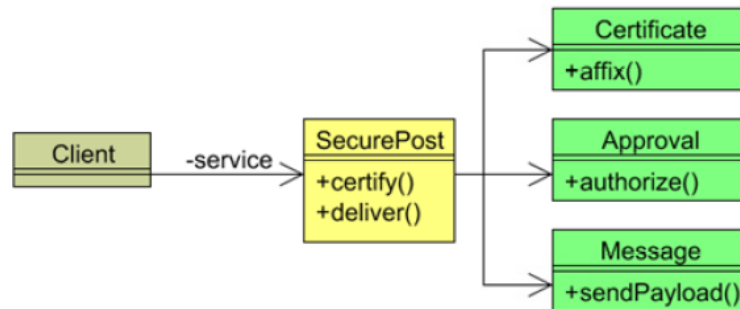
Singleton Pattern in UML



Singleton Pattern in UML

The Singleton pattern enforces a class that creates one instance of itself, which is returned by reference to all callers. Implementing this scheme can prove surprisingly complex if, for example, the data in the target class must be threadsafe, or available as single instance to multiple applications or a highly-available system, such as a cluster.

Facade Pattern in UML

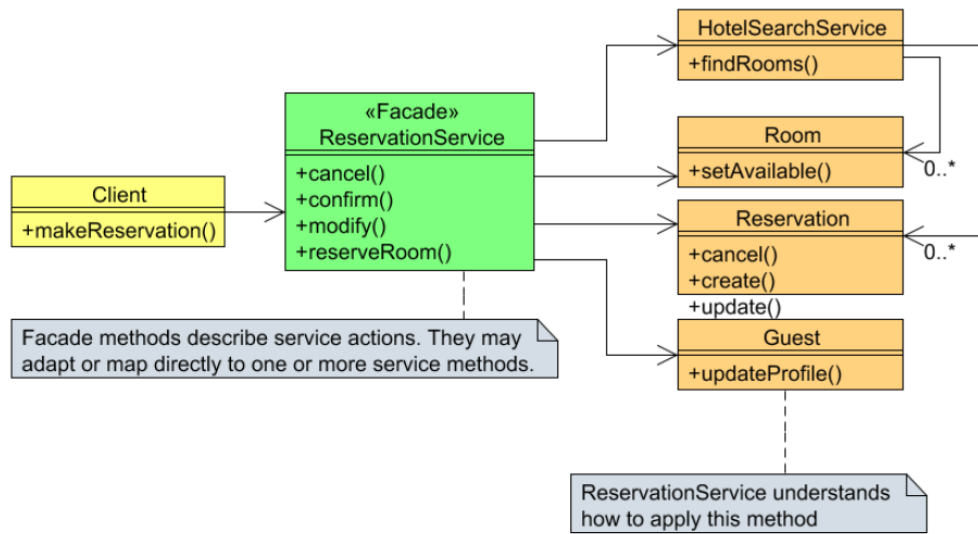


Facade Pattern in UML

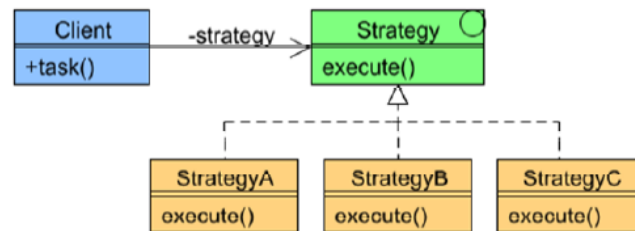
Facade is another pattern with an intuitive name. A Facade masks a complex process requiring multiple objects with a service-oriented client interface. The `SecurePost` class in the slide accesses a number of technical objects, provides for their association with each other, and supports the logic the client needs to complete a service-oriented task. At the same time, access to the `Certificate`, `Approval`, and `Message` classes is not hidden by the Facade. Other facades, or even so-called expert clients, can be developed later on.

Several related patterns, such as the Adapter, use the same approach to solve different problems.

Applying the Facade Pattern: Example Solution



Applying the Strategy Pattern: Structure

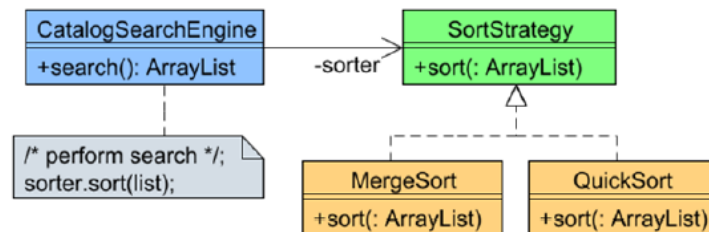


Applying the Strategy Pattern: Structure

The `Client` accesses a `Strategy` implementation through a compile-time `Strategy` reference. If default strategy is appropriate, realize it as a concrete class and subclass for each alternate algorithm or procedure.

In either case, the code that associates the `Client` and `Strategy` objects will instantiate the appropriate type and invoke its `execute()` method. Alternatively, a list of `Strategy` implementations could be exposed to the `Client` to choose from. Such an approach requires a more complex implementation that is not shown here.

Applying the Strategy Pattern: Example Solution



ORACLE

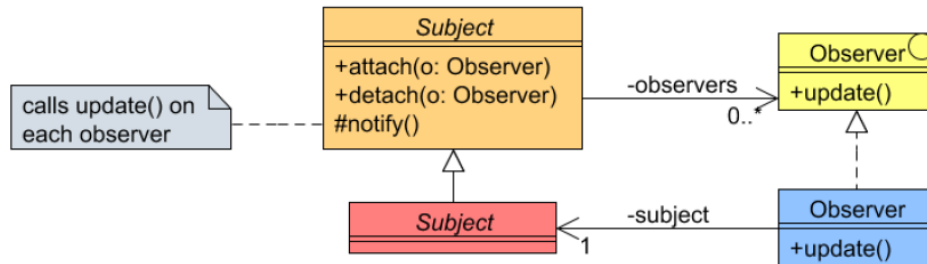
Copyright © 2011, Oracle and/or its affiliates. All rights reserved.

Applying the Strategy Pattern: Example Solution

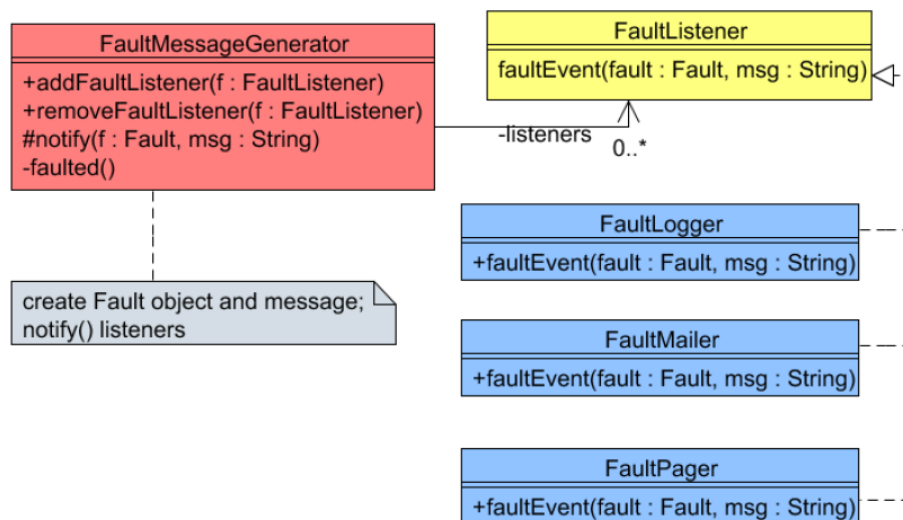
The `CatalogSearchEngine` class associates with a `SortStrategy` class that provides the default algorithm. When appropriate, the controlling code can instantiate a `MergeSort` or `QuickSort` object instead. Whichever object is instantiated, the subsequent step remains the same: call the `sort()` method to handle the details.

With this design pattern, the details of each method are separated from the decision logic. One outcome is cleaner decision code that is easier to read. Adding additional cases just requires a new subclass or implementation.

Applying the Observer Pattern: Structure



Applying the Observer Pattern: Example Solution



Patrones Java EE

Con la llegada de la plataforma J2EE para la creación de aplicaciones Web, se definió un catálogo de patrones específicos para la misma. Los patrones Java EE se dividen en cinco categorías, coincidiendo con las cinco capas en las que se puede estructurar una aplicación

- Cliente
- Presentación
- Negocios
- Integración
- Recurso

Los 15 patrones del catalogo se encuentran distribuidos entre las tres capas intermedias, Presentación, Negocios e Integración

Capa de presentación

Decorating Filter / Intercepting Filter	Un objeto que está entre el cliente y los componentes Web. Este procesa las peticiones y las respuestas.
Front Controller/ Front Component	Un objeto que acepta todos los requerimientos de un cliente y los direcciona a manejadores apropiados. El patrón Front Controller podría dividir la funcionalidad en 2 diferentes objetos: el <i>Front Controller</i> y el <i>Dispatcher</i> . En ese caso, El Front Controller acepta todos los requerimientos de un cliente y realiza la autenticación, y el Dispatcher direcciona los requerimientos a manejadores apropiada.
View Helper	Un objeto helper que encapsula la lógica de acceso a datos en beneficio de los componentes de la presentación. Por ejemplo, los <i>JavaBeans</i> pueden ser usados como patrón View Helper para las páginas JSP.
Composite view	Un objeto vista que está compuesto de otros objetos vista. Por ejemplo, una página <i>JSP</i> que incluye otras páginas <i>JSP</i> y <i>HTML</i> usando la directiva include o el action include es un patrón Composite View.
Service To Worker	Es como el patrón de diseño MVC con el Controlador actuando como Front Controller pero con una cosa importante: aquí el Dispatcher (el cual es parte del Front Controller) usa View Helpers a gran escala y ayuda en el manejo de la vista.
Dispatcher View	Es como el patrón de diseño MVC con el controlador actuando como Front Controller pero con un asunto importante: aquí el Dispatcher (el cual es parte del Front Controller) no usa View Helpers y realiza muy poco trabajo en el manejo de la vista. El manejo de la vista es manejado por los mismos componentes de la Vista.

Capa de Negocios

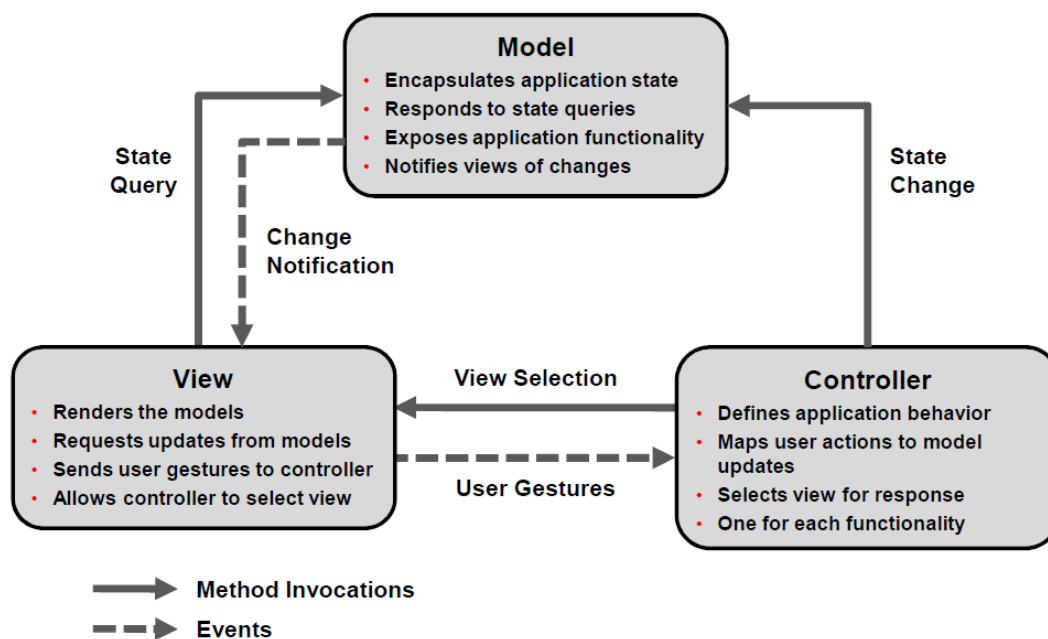
Business Delegate	Un objeto que reside en la capa de presentación y en beneficio de los otros componentes de la capa de presentación llama a métodos remotos en los objetos de la capa de negocios.
Value Object/ Data Transfer Object/ Replicate Object	Un objeto serializable para la transferencia de datos sobre la red.
Session Façade/ Session Entity Façade/ Distributed Façade	El uso de un bean de sesión como una fachada (facade) para encapsular la complejidad de las interacciones entre los objetos de negocio y participantes en un flujo de trabajo. El Session Façade maneja los objetos de negocio y proporciona un servicio de acceso uniforme a los clientes.
Aggregate Entity	Un bean entidad que es construido o es agregado a otros beans de entidad.
Value Object Assembler	Un objeto que reside en la capa de negocios y crea Value Objects cuando es requerido.
Value List Handler/ Page-by-Page Iterator/ Paged List	Es un objeto que maneja la ejecución de consultas SQL, caché y procesamiento del resultado. Usualmente implementado como beans de sesión.
Service Locator	Consiste en utilizar un objeto Service Locator para abstraer toda la utilización JNDI y para ocultar las complejidades de la creación del contexto inicial, de búsqueda de objetos home EJB y recreación de objetos EJB. Varios clientes pueden reutilizar el objeto Service Locator para reducir la complejidad del código, proporcionando un punto de control.

Capa de Integración

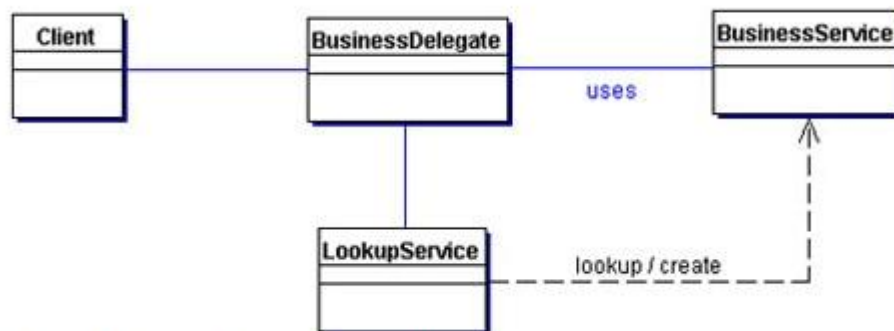
Data Access Object Service Activator	Consiste en utilizar un objeto de acceso a datos para abstraer y encapsular todos los accesos a la fuente de datos. El DAO maneja la conexión con la fuente de datos para obtener y almacenar datos.
Service Activator	Se utiliza para recibir peticiones y mensajes asíncronos de los clientes. Cuando se recibe un mensaje, el Service Activator localiza e invoca a los métodos de los componentes de negocio necesarios para cumplir la petición de forma asíncrona.

Patrón MVC

Applying the MVC Pattern: Structure



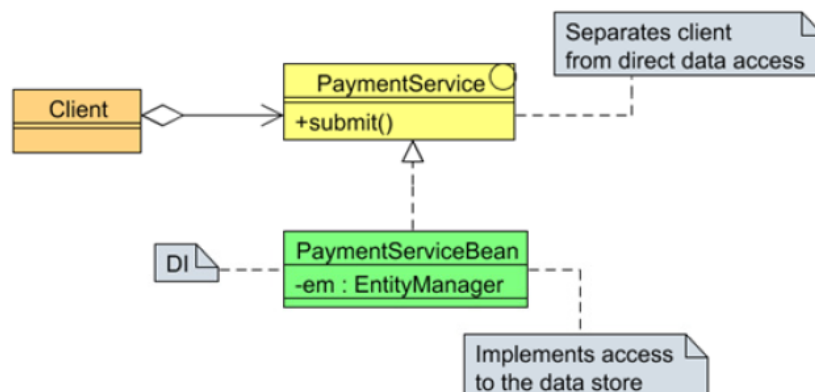
Patrón Business Delegate



Oculto a los clientes los detalles de acceso a la lógica de negocio. Utilizado antiguamente con el acceso a EJB 2

Patrón DAO

DAO Pattern



ORACLE

Copyright © 2011, Oracle and/or its affiliates. All rights reserved.

DAO Pattern

The **Client** uses a **PaymentService** interface that exposes a `submit()` service method. **PaymentServiceBean** handles the details of bringing data into the transaction scope by injecting an **EntityManager** reference and using it to access the database. The service-oriented logic is kept separate from the data access logic, thereby releasing the **Client** from having to participate or even be aware of the process.

Se apoya en el patrón de método de factoría:

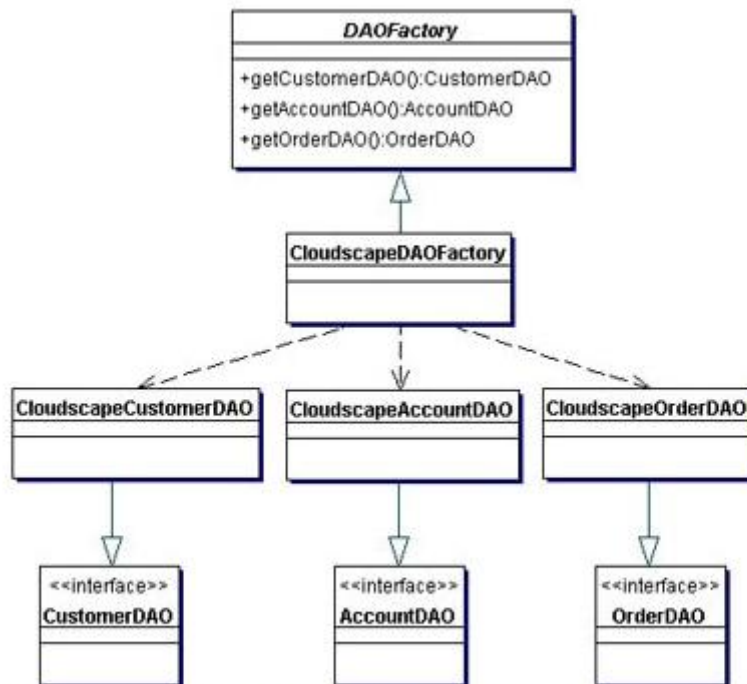


Figure 9.7 Implementing the Factory for DAO strategy using Factory Method

Y en el de factoría abstracta:

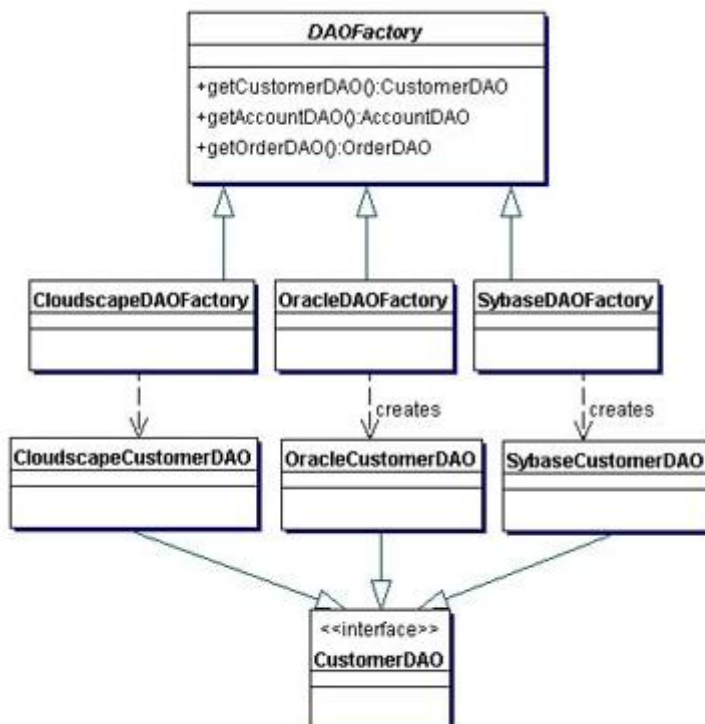
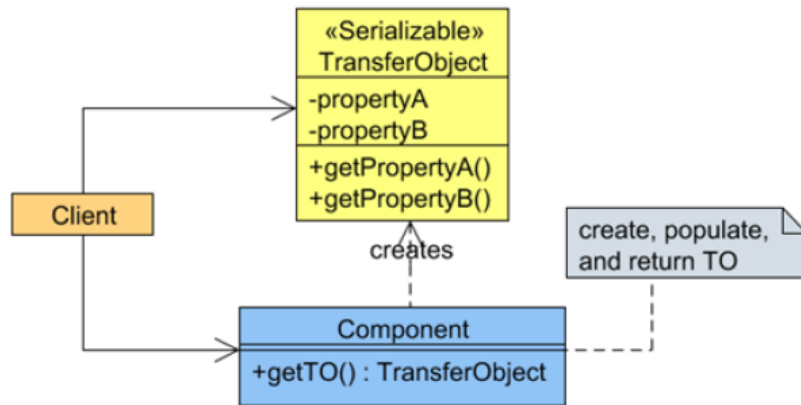


Figure 9.8 Implementing the Factory for DAO strategy using Abstract Factory Example 9.1
Abstract DAOFactory Class

Data Transfer Object Pattern Structure



Data Transfer Object Pattern Structure

The Client retrieves a **TransferObject** through a **Component** class. The **Component** class is responsible only for creating **TransferObject** and populating it with data from another object with a reference to the needed data. A **TransferObject** makes it possible to deliver data to any **Client** by foregoing a referenced connection to the data's source.

Patrón Composite View

Consiste en crear una plantilla de vista e incluir en ella otras subvistas, de modo que estos bloques o subvistas puedan ser reutilizados

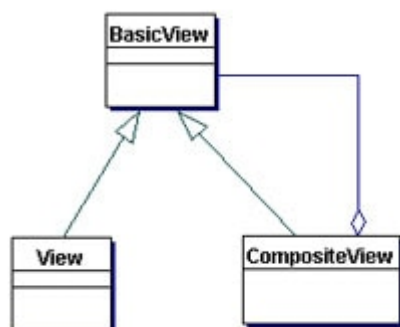


Figure 7.18 Composite View class diagram

Patrón View Helper

Consiste en delegar desde las vistas hacia clases helper de Java el procesamiento de datos. Estos helper se implementan mediante JavaBeans o acciones personalizadas.

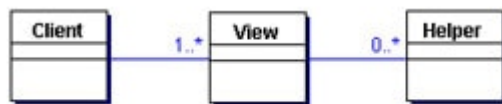


Figure 7.11 View Helper class diagram

Antipatrones

Un antipatrón representa una práctica que tiene un impacto negativo en el rendimiento y funcionamiento de un sistema.

Entre los antipatrones más conocidos están:

- Hot potato. Se produce en un entorno de mensajería. Ha servidores de mensajes que vuelven a enviar el mensaje hasta recibir un reconocimiento de su llegada. Si se produce una excepción y no hay reconocimiento, podemos encontrarnos con un envío permanente del mensaje. A estos es a lo que se conoce como hot potato
- Golden Hammer. Consiste en utilizar una determinada herramienta o práctica para muchas tareas a descripción, sin pensar que a lo mejor puede no ser adecuada para cierta tarea.
- Información de navegación embebida. Las páginas incluyen enlaces con direcciones a otras páginas. Se debe centralizar la navegación en front controller y utilizar direcciones lógicas